

ThomysHomeAgent

Konzept für den Betrieb auf einem Raspberry Pi

Ziel ist eine zentrale Smart-Home-Plattform, die dauerhaft auf einem Raspberry Pi läuft. Android-Smartphones und Tablets dienen als Bedienoberflächen. Der Raspberry Pi übernimmt Server, Automatisierung, Zigbee2MQTT, MQTT und die Kommunikation mit den Smart-Home-Geräten.

1. Zielarchitektur

Android / Tablet

ThomysHome App als installierbare PWA

↓ WLAN / LAN

Raspberry Pi

ThomysHomeAgent • lichtapp.py • lichtagent.py • HomeBrain • REST API

Zigbee2MQTT • MQTT Broker • optional Home Assistant

↓

Zigbee USB-Stick → Licht • Taster • Sensoren • Aktoren

2. Aufgaben des Raspberry Pi

Komponente	Aufgabe
ThomysHomeAgent	Zentrale Benutzeroberfläche und Steuerungslogik
lichtapp.py	Web-App, REST-API und Bedienoberfläche
lichtagent.py	HomeBrain, Automatisierung und Gerätesteuerung
Zigbee2MQTT	Kommunikation mit Zigbee-Geräten
MQTT	Nachrichtenübertragung zwischen Komponenten
Home Assistant (optional)	Integration weiterer Smart-Home-Systeme
PWA	App-Oberfläche für Android ohne klassischen App-Store

3. Projektstruktur

~/lichtagent/

■■■■ lichtapp.py

■■■■ lichtagent.py

■■■■ pwa/

■ ■■■■ manifest.json

■ ■■■■ service-worker.js

■ ■■■■ icons/

■■■■ templates/

■ ■■■■ dashboard.html

■■■■ static/

■ ■■■■ css/

■ ■■■■ js/

- ■■■■ icons/
- settings.json
- scenes.json
- layout.json
- config.json (geheim, nicht teilen)

4. Kommunikation

Die Android-App soll nicht direkt mit Zigbee2MQTT oder dem MQTT-Broker kommunizieren. Alle Steuerbefehle laufen über ThomysHomeAgent auf dem Raspberry Pi. Dadurch bleiben Zugangsdaten, MQTT-Passwort und Home-Assistant-Token auf dem Server.

5. Android-App

Die Benutzeroberfläche wird als Progressive Web App (PWA) bereitgestellt. Sie kann auf Android zum Startbildschirm hinzugefügt werden, erhält ein eigenes Icon und kann im App-ähnlichen Vollbild laufen. Das Smartphone ist damit der Client; der Raspberry Pi ist der zentrale Server.

Beispiel: <http://raspberrypi:8099> oder später eine lokale HTTPS-Adresse wie <https://home.thomys.local>

6. Sicherheit

Die Datei **config.json** darf nicht in ein öffentliches Repository oder in einen Chat hochgeladen werden, wenn sie HA-Tokens oder MQTT-Passwörter enthält. Geheimnisse bleiben ausschließlich auf dem Raspberry Pi. Für einen späteren produktiven Betrieb sollte die Web-App über HTTPS und mit einer geeigneten Authentifizierung abgesichert werden.

7. Geplanter Funktionsumfang

- Häuser und Standorte
- Stockwerke und Räume
- Zigbee-Geräte und Gerätestatus
- Lichtsteuerung
- Szenen und Automatisierungen
- HomeBrain für intelligente Steuerungen
- Zigbee2MQTT-Status und Geräteinformationen
- Info-Panel mit System- und Zigbee2MQTT-Informationen
- REST-Endpunkte unter `/api/z2m/*`
- Android-PWA als zentrale Bedien-App

8. Umsetzung in sinnvoller Reihenfolge

1. Bestehende lichtapp.py und lichtagent.py analysieren.
2. ThomysHomeAgent auf dem Raspberry Pi lauffähig machen.
3. Zigbee2MQTT und MQTT anbinden.
4. Zigbee-Ziele in HomeBrain integrieren.
5. REST-API `/api/z2m/*` ergänzen.

6. PWA in die bestehende Web-App integrieren.
7. Autostart als Systemdienst einrichten.
8. Android-App installieren und im Heimnetz testen.
9. Optional HTTPS, Benutzeranmeldung und Remote-Zugriff ergänzen.

9. Zielbild

Der Raspberry Pi wird zur zentralen ThomysHome-Appliance. Nach dem Einschalten startet die Smart-Home-Software automatisch. Im Heimnetz öffnen Benutzer ThomysHome auf Android oder Tablet. Die Bedienung erfolgt über eine einheitliche Oberfläche, während ThomysHomeAgent im Hintergrund Home Assistant, MQTT und Zigbee2MQTT koordiniert.